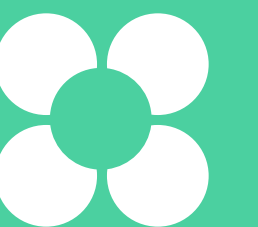


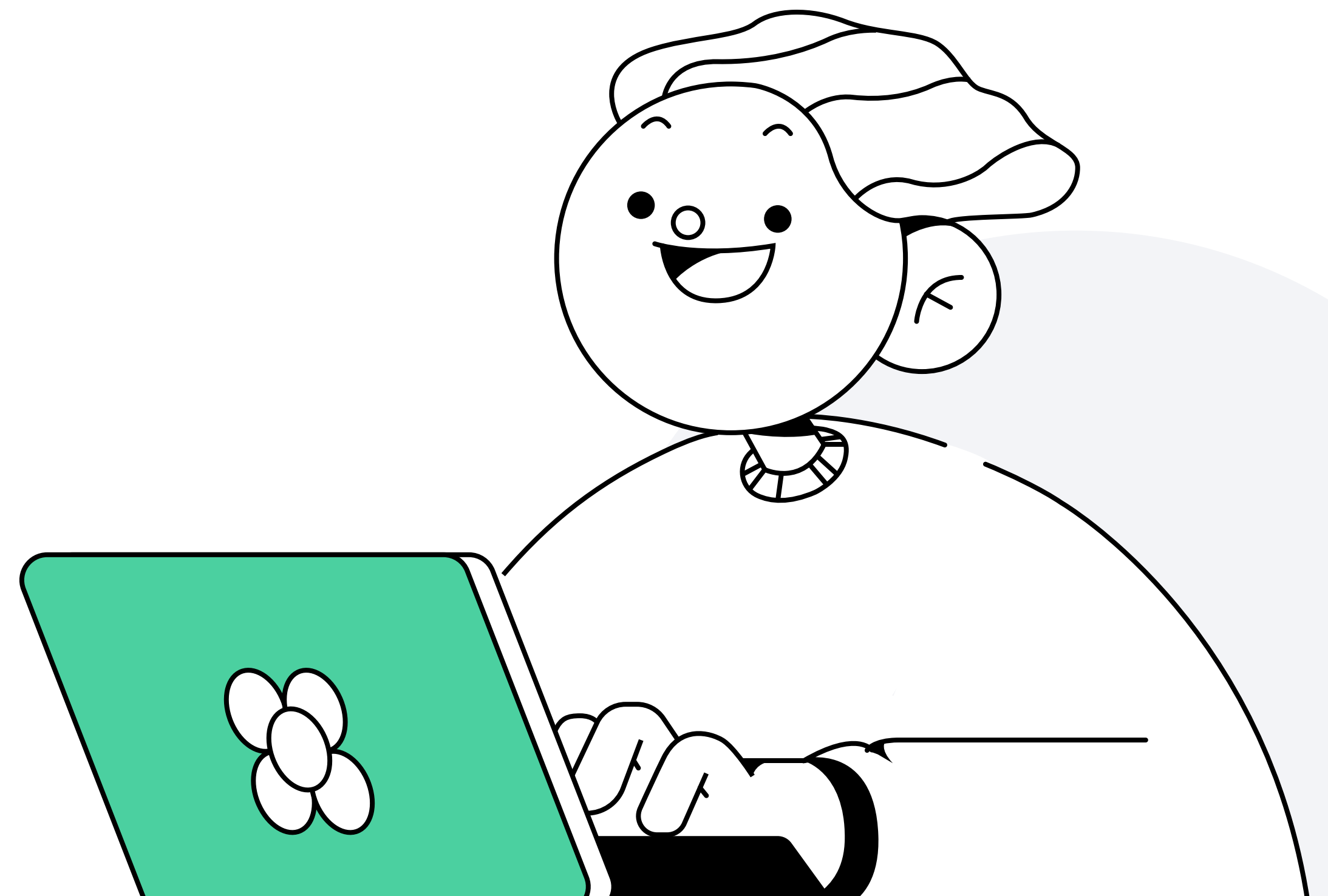
Практическое знакомство с Jenkins

Алексей Метляков
Tech Lead DevOps engineer в OpenWay



План занятия

- 1 Jenkins
- 2 Сущности Jenkins
- 3 Дополнения



Jenkins

Алексей Метляков

Tech Lead DevOps engineer в OpenWay

Что такое Jenkins

Jenkins – ППО для управления **CI\CD**-процессами

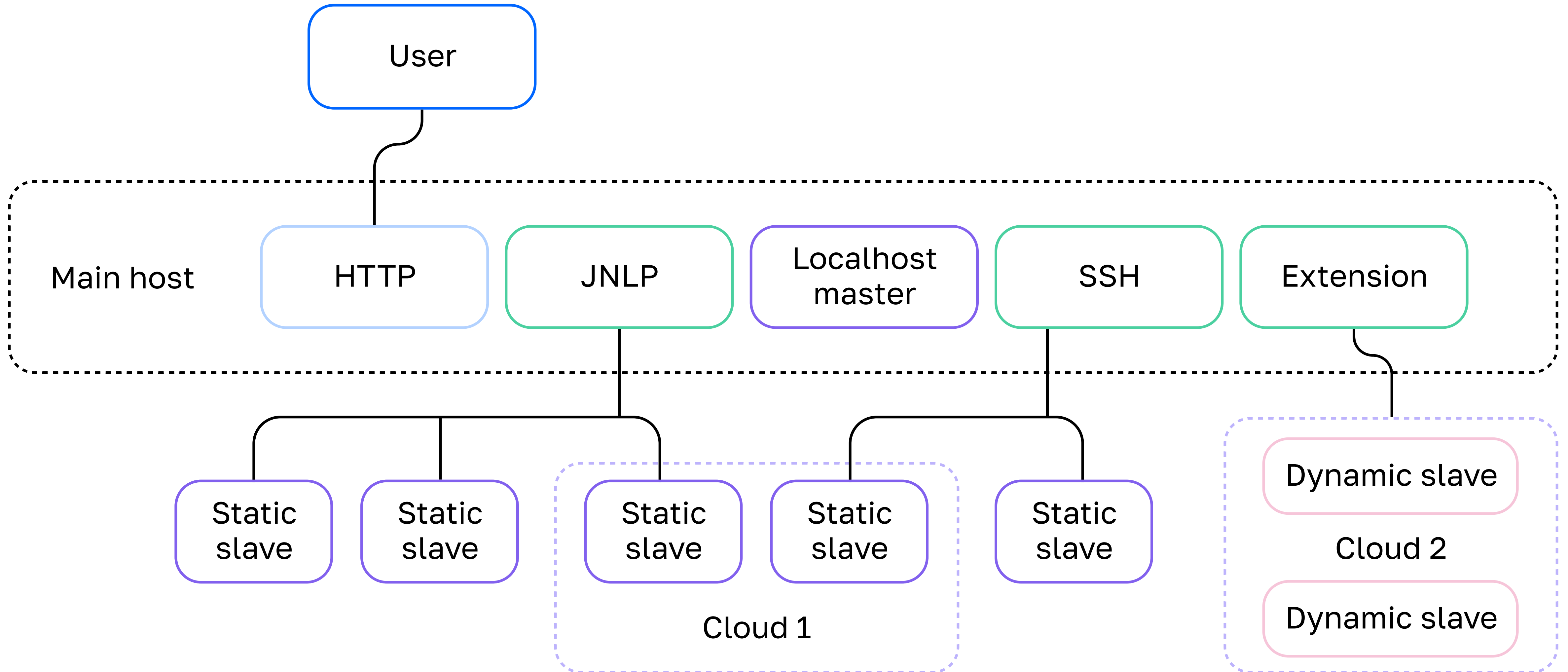
- **Open source** – следовательно, бесплатен
- **Многофункционален** – большое количество видов использования автоматизаций
- Возможность **расширения**
- Использует встроенные синтаксисы: **Groovy** и **Pipeline**

Что такое Jenkins

Некоторые основные возможности системы:

- **Создание** конвейера **автоматизации**
- Создание пользователей и **управление** ими
- Синхронизация с **AD**
- Ограничение прав пользователей на разных вложенностях сущностей
- Установка **плагинов**
- Настройка **Custom Tool**
- Использование динамических и статических окружений

Архитектура Jenkins



Сущности Jenkins

Алексей Метляков

Tech Lead DevOps engineer в OpenWay

Freestyle Job

Freestyle Job – простейшее описание рутины в виде последовательно выполняемых заданий.

- Задача может быть описана удобным способом
- Часть заданий можно описать в репозитории

Groovy

Groovy – объектно-ориентированный язык программирования, разработанный как дополнение к **Java**.

- Apache 2 **License**
- Может быть языком **статической** типизации
- Может быть языком **динамической** типизации
- Имеет **встроенный** синтаксис для работы с массивами, списками и регулярками
- Считается смесью **Java** и **Python**, **Ruby** и **Smalltalk**

Больше о самом языке на [официальном сайте](#)

Groovy

В контексте **Jenkins Groovy** может нас заинтересовать следующими объектами:

- Package **jenkins**
 - jenkins.*
 - jenkins.model.*
- Package **hudson**
 - hudson.*
 - hudson.model.*

```
Jenkins.instance.computers.each{  
    println "${it.displayName} ${it.hostName}"  
}
```

Для полноценного использования можно ознакомиться с содержимым [официальной страницы](#)

Pipeline Job

Pipeline Job – описание рутины в отдельных файлах с **Pipeline** на собственном синтаксисе.

Может быть описано в двух видах:

- **Declarative Pipeline**
- **Scripted Pipeline**

Pipeline Syntax

Declarative Pipeline – верхнеуровневое описание того, что необходимо сделать. Имеет свой собственный синтаксис и описывается в файлах с именем **Jenkinsfile**



Pipeline Syntax

Пример **Declarative Pipeline**:

```
pipeline {  
    agent any  
    stages {  
        stage('Build'){  
            steps{  
            }  
        }  
        stage('Test'){  
            steps{  
            }  
        }  
    }  
}
```

Pipeline Syntax

Scripted Pipeline – описание стадий конвейера с использованием собственного синтаксиса, но с дополнениями в виде **Groovy-скриптов**



Pipeline Syntax

Пример **Scripted Pipeline**:

```
node {  
    stage('Build'){  
    }  
  
    stage('Test'){  
        if (currentBuild.currentResult == SUCCESS){  
            stage('Test'){  
            }  
        }  
    }  
}
```

Pipeline Syntax

- **Jenkins** был написан в тесной связи с **Groovy**
- **Groovy** нужно было изолировать от использования системных вызовов — был создан **Scripted Pipeline**
- Чтобы не принуждать всех пользователей изучать **Groovy**, был создан **Declarative Pipeline** с упрощённым интерфейсом

Полное описание работы с Pipeline можно найти в [документации](#)

Multibranch Pipeline

Multibranch Pipeline — вид Pipeline, который умеет запускать сборки по действиям в одном репозитории

- Умеет разделять виды действий в репозитории
- Фильтровать имена branches

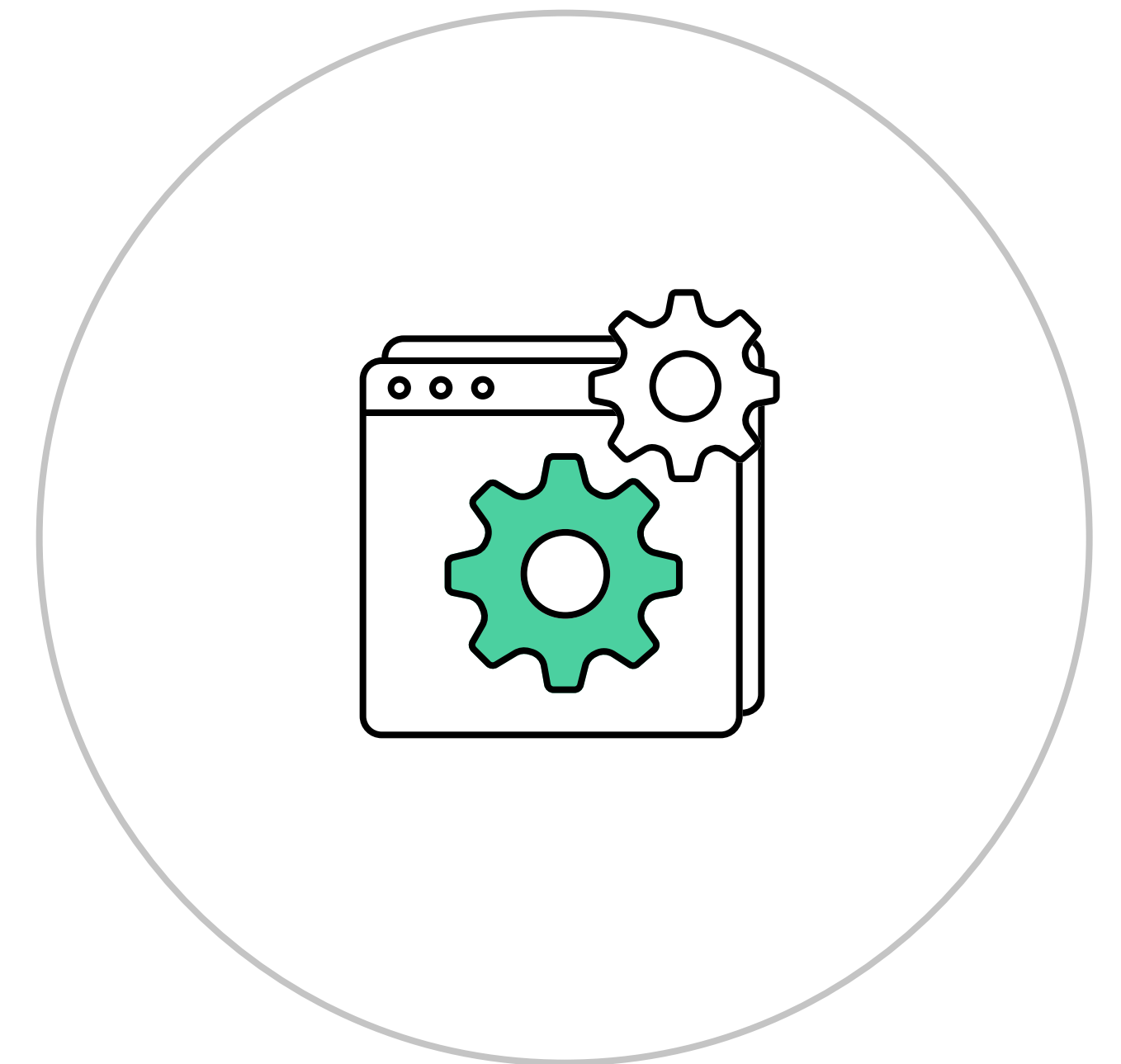
Дополнения

Алексей Метляков

Tech Lead DevOps engineer в OpenWay

Blue Ocean

Blue Ocean – новый UI для Jenkins, который призван улучшить читаемость происходящего и сделать более качественную визуализацию прохождения этапов сборки



Ansible

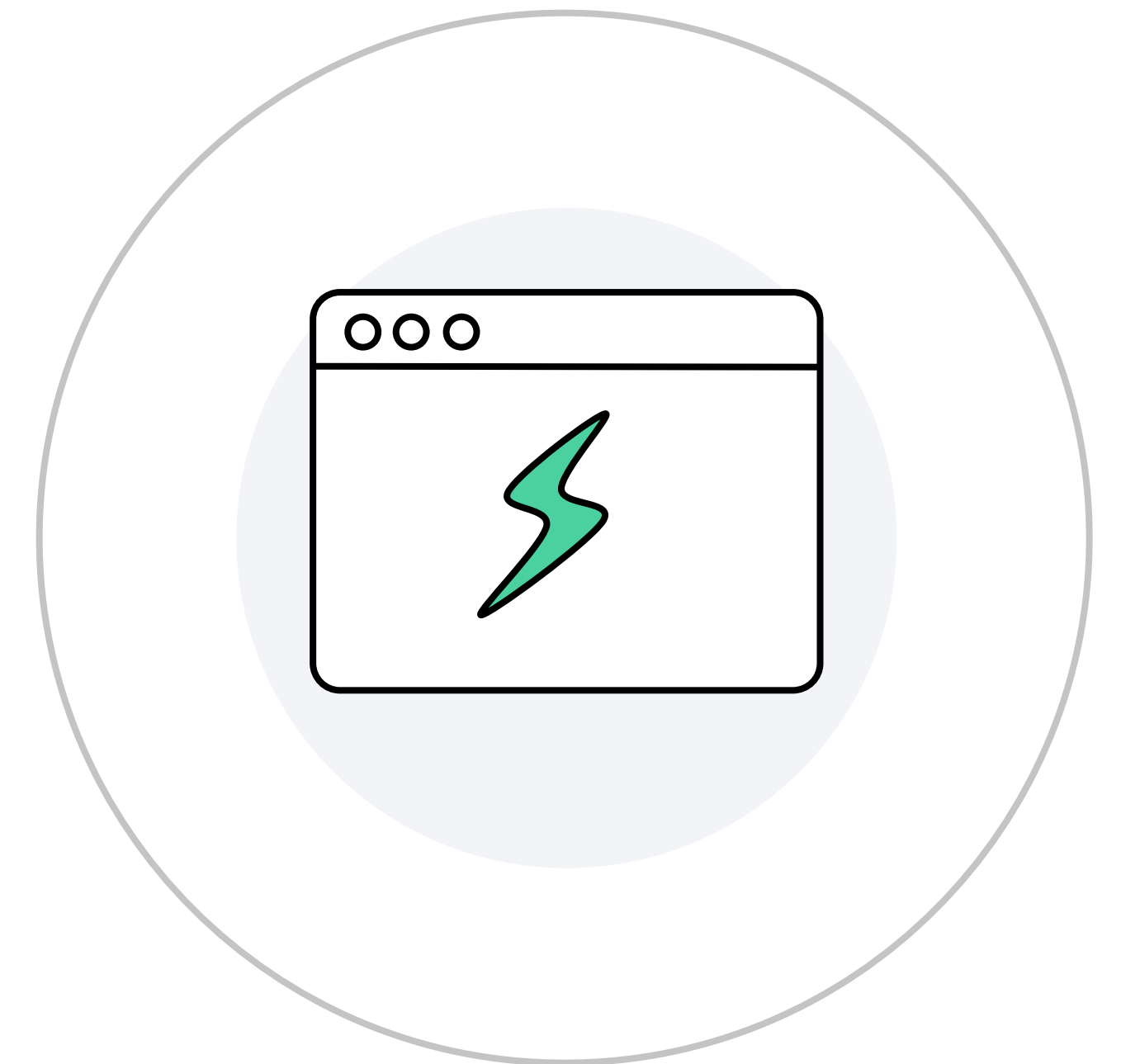
Ansible – добавляет возможность запускать Ansible через синтаксис Declarative Pipeline. Фактически избавляет от необходимости вызова оболочки для вызова плейбука.

Пример синтаксиса можно посмотреть на [официальной странице плагина](#):

```
ansibleColor('xterm') {  
  ansiblePlaybook(  
    playbook: 'path/to/playbook.yml',  
    inventory: 'path/to/inventory.ini',  
    credentialsId: 'sample-ssh-key',  
    colored: true)  
}
```

Monitoring

Monitoring – плагин, позволяющий отслеживать Java-процессы через **JavaMelody**. Можно отслеживать как процессы **master**, так и **agents**



Monitoring

Monitoring позволяет:

- проверять **работоспособность** потоков
- посмотреть созданные объекты в **heap**
- вызывать **GC**
- собирать **heapdump**